

Tejas Nikhar

📍 Bengaluru, Karnataka, India ✉ tejastn10@gmail.com ☎ +91 8928088153 📧 in/tejastn10 🌐 tejastn10.com

SUMMARY

Backend engineer and Technical Lead with 5+ years designing and scaling distributed systems on AWS. Leads platform architecture spanning multi-service data migrations, production incident recovery under live traffic, and org-wide standards for serverless scaffolding, feature-flagged rollouts, and CI/CD. Strong across Node.js, NestJS, and Golang, with depth in authentication, rate limiting, SOC2 compliance, and observability.

EXPERIENCE

Technical Lead | Habuild

May 2025 - Present, Bangalore, India

- **Diagnosed and resolved a production outage** from Postgres connection exhaustion on a shared primary crashing ECS tasks across services — rerouted **60–70% of read traffic** to idle replicas overnight, restoring reads to **p50 2–5 ms / p95 15–26 ms** across major services, and extended the same replica pattern to Redis.
- **Led** a cross-service member-unification migration consolidating **3 legacy member tables and 35+ downstream tables across 5 microservices** into a single **UUID-keyed accounts + members model** — backfilling **5M+ records** through a **dual-write, feature-flag-gated phased cutover** with **zero downtime**, ending in legacy archival.
- **Designed and shipped** a Lambda-based OTP delivery system with isolated senders per channel (WhatsApp via WABA, SMS via gateway) and per environment — sustaining peak volumes of **~1M+ OTPs** at a **0% system-side error rate** and **97% end-to-end delivery**.
- **Overhauled** the platform authentication layer and unified registration for all frontends (free and paid) into a **single source of truth** — eliminating duplicate account creation and adding a **Redis-backed rate-limiting layer** to secure previously open endpoints.
- **Re-architected** the free-member app flow to close a **data-exposure risk** in the registration API, splitting one endpoint into a frictionless public challenge-registration flow and an **OTP-gated, rate-limited** dashboard-login flow — enabling personalized journeys for repeat-challenge members.
- **Introduced GrowthBook** as the org's unified feature-flag and experimentation platform, gating API rollouts (legacy → v2 cutovers, dual-write toggles, percentage-based read migration) at the NestJS controller level — **now the platform standard**.
- **Architected** a **Docker-based Lambda scaffold** (Node.js and Python templates with local SQS / API Gateway emulation) adopted as the **org-wide standard** for all containerized serverless functions, versioned and deployed via ECR.
- **Built** a production-grade **NestJS service template** and a centralized file service that removed duplicated S3 client code, and consolidated GitHub Actions into an **org-level shared-workflows repo**.
- **Rebuilt** the source-attribution pipeline for free-member registration and reactivation flows in partnership with Marketing — restoring attribution across **web, mobile-app, and Truecaller** surfaces after RCA.
- **Established** branch-based dev deployments via GitHub Actions, standardized AWS resource-naming conventions, and **launched a new Link service in Golang** before transitioning ownership.

Software Development Engineer II | Rocketium

July 2022 - April 2025, Bangalore, India

- **Architected** a **BullMQ** queue-driven export pipeline running **Puppeteer across parallel Lambda workers**, sustaining **500K+ creatives/day** across **10+ organizations** with real-time status, retry handling, and database updates on every completed job.
- **Led** remediation of **800+ platform vulnerabilities** to achieve **SOC2 compliance**, strengthening security posture across all services.
- **Migrated 65+ AWS Lambda functions** (Node.js 14 → 18 → 20) with all dependent packages, and **authored a custom GitHub Action** for CI/CD deployment of Docker-based Lambdas with automatic IAM enforcement.
- **Containerized** service deployments (AWS ECS/ECR) and built GitHub Actions CI/CD pipelines, cutting deployment time from **20 minutes to 5 minutes (~75% faster)**.
- **Engineered** a **Brand Safety** system moderating GIFs and videos on Lambda — combining Google Vision for frame-by-frame content analysis with a real-time rules engine enforcing **30+ workspace-specific brand rules** on every creative.
- **Automated** canvas image export via Puppeteer-in-Lambda with optimistic credit computation and a stuck-project recovery mechanism, cutting export time by **2m 45s** on benchmarked **5,000-creative projects**.
- **Designed** a scalable version-history system on S3 + Lambda using **Brotli compression** for **40% volume reduction**, paired with S3 lifecycle policies that auto-expire outdated archives to cut long-term storage costs.
- **Built** Rocketium's core video editor (comparable to Adobe After Effects), implementing precise timeline manipulation, keyframe movement, and layer snapping to support video production at scale.

Software Development Engineer I | Rocketium

May 2021 - July 2022, Bangalore, India

- **Cut** analytics processing from **several minutes to 10–15 seconds** by relocating compute-heavy front-end calculations to the backend, in partnership with backend engineers.
 - **Developed** a new animation engine on the **Web Animations API (WAAP)**, delivering stable **50+ FPS** playback across browsers for all projects and improving render performance by **40%**.
 - **Established** CI hygiene across **10+ repositories** (Husky, Commitlint, PR templates, code review) and a unified logging system consolidating frontend and backend data, **resolving 50+ critical on-call issues**.
 - **Engineered** custom data-filter solutions for the Creative Analytics platform serving enterprise clients including **Airtel, ADA, and Rappi**, streamlining analysis across large creative datasets.
 - **Instituted** a fail-fast approach to front-end data handling, surfacing malformed or missing data at the point of failure and shortening issue-resolution cycles.
 - **Extended** the video editor's property and styling system to video elements and rebuilt interface components, including a modular, context-aware context-menu system.
-

PROJECT

Nest Pingbot

www.npmjs.com/package/nest-pingbot

- Published nest-pingbot, an open-source NestJS module on NPM that unifies Slack and Discord messaging behind a single sendMessage interface, shipping **8 versioned releases** with **100% TypeScript coverage**.
- Implemented platform-specific capabilities including Slack Block Kit UI, thread replies, Discord embeds, and scheduled message delivery, with graceful error handling for unconfigured platforms.
- Designed **3 module registration patterns** (basic, global, and async factory-based) to support any NestJS app and config setup, and automated quality gates with GitHub Actions, Husky, Commitlint, and BiomeJS.

AWS-Lambda-Radar

www.npmjs.com/package/aws-lambda-radar

- Published aws-lambda-radar, an open-source TypeScript middleware on NPM that wraps Lambda handlers to standardize logging and error handling, eliminating repetitive boilerplate across serverless functions.
- Shipped **6 versioned releases** with CI-driven quality checks, packaging the middleware for drop-in adoption via both npm and Yarn.
- Automated context capture (request IDs, function metadata, invocation details) on every invocation, producing structured CloudWatch logs that improve traceability during production debugging.

Cortex

github.com/Kosha-Nirman/cortex

- Built Cortex, a subdomain reconnaissance engine in Golang that aggregates **5 discovery techniques** (DNS brute force, certificate transparency logs, search-engine dorking, third-party APIs, and passive archive collection) into a single scan.
- Engineered a concurrent scanning pipeline with adaptive rate limiting and request throttling, enabling high-throughput enumeration while applying stealth evasion to avoid detection.
- Designed a modular enumeration architecture that auto-generates Markdown reports, breaking down active/inactive subdomains, per-source statistics, and security recommendations.

Gaia

github.com/tejastn10/gaia

- Built Gaia, a Golang CLI that rewrites the first commit message in a Git repository via automated interactive rebase, collapsing a multi-step manual workflow into a single command across **3 released versions**.
- Implemented **4 safety layers** before any history rewrite (repository validation, clean-working-tree verification, explicit force-push confirmation, and graceful rebase-failure recovery) to protect commit integrity.
- Automated distribution with a one-line curl installer and matching uninstall script, backed by GitHub Actions workflows for unit testing and release publishing.

EDUCATION

B.Tech in Computer Engineering | Datta Meghe College of Engineering (University of Mumbai)

8.8, Mumbai, India, 2017-2021

SKILLS

Programming Languages: JavaScript, TypeScript, Python, Golang, SQL, NoSQL

Backend & APIs: Node.js, NestJS, Express, REST APIs, BullMQ, json-rules-engine, event-driven architecture, distributed systems

Frontend: React, React Native, Redux, Redux-Saga, Web Animations API

Cloud & DevOps: AWS (Lambda, S3, SNS, ECS, ECR, API Gateway, SQS), Docker, GitHub Actions (CI/CD), Git

Databases & Storage: PostgreSQL (primary/replica), MongoDB, Redis, AWS Redshift, BigQuery, Amazon S3

Monitoring & Observability: SigNoz, New Relic, OpenTelemetry, Amazon CloudWatch

Testing & Quality: Jest, Cypress, Husky, Commitlint, unit and integration testing

Practices & Tools: Feature flags (GrowthBook), rate limiting, SOC2 compliance, Puppeteer, Google Vision API, Figma
